SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

---

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO* (BL3, BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

Annexure A
Coding Challenge

Code of Conduct:

I K PHANI SRIDHAR YOGI / 192311091(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Coding Challenge

Coding Challenge (Additional Set)

1. Write a Python program to simulate **Secure Email Transmission using S/MIME**, including message encryption, digital signature generation, and verification. Analyze how confidentiality and authentication are ensured.
2. Develop a Python application to implement **PGP-based file encryption and decryption**, including key generation, key distribution, and message authentication. Evaluate how PGP provides both security and efficiency.
3. Implement a Python program to simulate **IPSec Tunnel Mode**, encrypting an entire IP packet using symmetric encryption. Analyze how tunnel mode differs from transport mode in securing network communication.
4. Write a Python script to implement **SSL/TLS handshake simulation**, including certificate exchange, session key generation, and secure data transfer. Evaluate how the handshake ensures secure web communication.
5. Build a Python-based tool to perform **Email Phishing Detection**, using feature extraction (URL patterns, headers, keywords) and classification techniques. Analyze the effectiveness of the model in identifying malicious emails.

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficien t approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Question 1: Secure Email Transmission using S/MIME

Aim

To write a Python program to simulate secure email transmission using S/MIME by implementing message encryption, digital signature generation, and signature verification.

Algorithm

1. Read the email message from the sender.
2. Generate a digital signature for the message using the sender's private key.
3. Encrypt the message using the receiver's public key.
4. Transmit the encrypted message and digital signature.
5. At the receiver side, decrypt the message using the receiver's private key.
6. Verify the digital signature using the sender's public key.
7. Display whether authentication and integrity verification are successful.

Python Program

```
import hashlib
import base64

# Simulated keys
sender_private_key = "SENDER_PRIVATE_KEY"
sender_public_key = "SENDER_PUBLIC_KEY"
receiver_public_key = "RECEIVER_PUBLIC_KEY"
receiver_private_key = "RECEIVER_PRIVATE_KEY"


# Encryption function
def encrypt_message(message, key):
    encrypted = []

    for i, char in enumerate(message):
        encrypted.append(
            chr(ord(char) ^ ord(key[i % len(key)]))
        )

    return ''.join(encrypted)


# Decryption function
def decrypt_message(encrypted_message, key):
    return encrypt_message(encrypted_message, key)


# Generate digital signature
def generate_signature(message, private_key):
    data = message + private_key
    return hashlib.sha256(data.encode()).hexdigest()


# Verify digital signature
def verify_signature(message, signature):
```

```
    expected = hashlib.sha256(
        (message + sender_private_key).encode()
    ).hexdigest()

    return expected == signature


# Sender side
message = input("Enter the email message: ")

print("\nOriginal Message:")
print(message)

# Generate digital signature
signature = generate_signature(
    message,
    sender_private_key
)

print("\nDigital Signature:")
print(signature)

# Encrypt message
encrypted_message = encrypt_message(
    message,
    receiver_public_key
)

# Encode encrypted message for transmission
encoded_message = base64.b64encode(
    encrypted_message.encode('latin1')
).decode()

print("\nEncrypted Message:")
print(encoded_message)

print("\n--- Email Sent Securely ---")


# Receiver side
decoded_message = base64.b64decode(
    encoded_message
).decode('latin1')

# Decrypt message
decrypted_message = decrypt_message(
    decoded_message,
    receiver_public_key
```

```
)

print("\nDecrypted Message:")
print(decrypted_message)

# Verify signature
is_valid = verify_signature(
    decrypted_message,
    signature
)

print("\nDigital Signature Verification:")

if is_valid:
    print("Signature Verified Successfully")
    print("Authentication: Successful")
else:
    print("Signature Verification Failed")
    print("Authentication: Failed")


# Security analysis
print("\n--- Security Analysis ---")

print("1. Confidentiality:")
print("The email message is encrypted before transmission.")

print("2. Authentication:")
print("The sender's digital signature is verified by the receiver.")

print("3. Integrity:")
print("Any modification to the message will cause signature verification to fail.")
```

Sample Output

```
Enter the email message: Hello, this is a secure email.

Original Message:
Hello, this is a secure email.

Digital Signature:
8c7...

Encrypted Message:
...

--- Email Sent Securely ---

Decrypted Message:
Hello, this is a secure email.
```

Digital Signature Verification:
Signature Verified Successfully
Authentication: Successful

--- Security Analysis ---
1. Confidentiality:
The email message is encrypted before transmission.

2. Authentication:
The sender's digital signature is verified by the receiver.

3. Integrity:
Any modification to the message will cause signature verification to fail.
Analysis
S/MIME provides confidentiality by encrypting the email message so that only the intended receiver can decrypt it. Authentication is achieved through the sender's digital signature, which allows the receiver to verify the sender's identity. The digital signature also provides integrity, because modification of the message causes signature verification to fail.
The above program is a simulation of the S/MIME process for demonstrating the concepts of encryption, digital signatures, authentication, and integrity.

Question 2: PGP-Based File Encryption and Decryption
Aim
To develop a Python application to implement PGP-based file encryption and decryption, including key generation, key distribution, and message authentication.
Algorithm

1. Generate a public key and private key for the user.
2. Store the keys securely.
3. Select the file to be encrypted.
4. Encrypt the file using the recipient's public key.
5. Generate a hash-based message authentication code for the file.
6. Decrypt the file using the recipient's private key.
7. Verify the message authentication code.
8. Display the decrypted file and authentication status.

Python Program

```
import hashlib
import base64

# Generate simulated PGP keys
public_key = "PGP_PUBLIC_KEY"
private_key = "PGP_PRIVATE_KEY"


# Encrypt file data
```

```
def encrypt_data(data, key):
    encrypted = []

    for i, char in enumerate(data):
        encrypted.append(
            chr(ord(char) ^ ord(key[i % len(key)]))
        )

    return ''.join(encrypted)


# Decrypt file data
def decrypt_data(data, key):
    return encrypt_data(data, key)


# Generate message authentication hash
def generate_hash(data):
    return hashlib.sha256(data.encode()).hexdigest()


# Create sample file
filename = "message.txt"

with open(filename, "w") as file:
    file.write("This is a confidential PGP protected file.")


# Read file
with open(filename, "r") as file:
    original_data = file.read()

print("Original File Content:")
print(original_data)


# Generate authentication hash
original_hash = generate_hash(original_data)

print("\nOriginal Hash:")
print(original_hash)


# Encrypt file
encrypted_data = encrypt_data(
    original_data,
    public_key
)
```

```
encoded_data = base64.b64encode(
    encrypted_data.encode('latin1')
).decode()


# Save encrypted file
with open("message.pgp", "w") as file:
    file.write(encoded_data)

print("\nFile encrypted successfully.")
print("Encrypted file: message.pgp")


# Decrypt file
with open("message.pgp", "r") as file:
    encoded_encrypted_data = file.read()

decoded_data = base64.b64decode(
    encoded_encrypted_data
).decode('latin1')

decrypted_data = decrypt_data(
    decoded_data,
    public_key
)


# Save decrypted file
with open("decrypted_message.txt", "w") as file:
    file.write(decrypted_data)

print("\nDecrypted File Content:")
print(decrypted_data)


# Verify authentication
decrypted_hash = generate_hash(decrypted_data)

print("\nDecrypted Hash:")
print(decrypted_hash)

if original_hash == decrypted_hash:
    print("\nMessage Authentication: Successful")
    print("File integrity verified.")
else:
    print("\nMessage Authentication: Failed")
    print("File integrity compromised.")
```

Sample Output

Original File Content:
This is a confidential PGP protected file.

Original Hash:
a91c...

File encrypted successfully.
Encrypted file: message.pgp

Decrypted File Content:
This is a confidential PGP protected file.

Decrypted Hash:
a91c...

Message Authentication: Successful
File integrity verified.

Analysis

PGP provides security by combining encryption and authentication techniques. The public key is used for encryption, while the corresponding private key is intended for decryption. A hash is generated to verify that the file has not been modified during transmission.
PGP provides **confidentiality** through encryption, **authentication** through key-based verification, and **integrity** through message hashing. It is also efficient because symmetric encryption can be used for the actual file data while public-key cryptography is used for secure key exchange.

Question 3: IPSec Tunnel Mode Simulation

Aim

To implement a Python program to simulate IPSec Tunnel Mode by encrypting an entire IP packet using symmetric encryption and to analyze the difference between Tunnel Mode and Transport Mode.

Algorithm

1. Create a sample IP packet containing source IP, destination IP, and data.
2. Apply symmetric encryption to the complete IP packet.
3. Encapsulate the encrypted packet as an IPSec Tunnel Mode packet.
4. At the receiver side, decrypt the packet.
5. Extract the original IP packet.
6. Display the original and decrypted packets.
7. Compare IPSec Tunnel Mode with Transport Mode.

Python Program

```
import base64

# Symmetric encryption key
key = "IPSEC_KEY"
```

```
# Encryption function
def encrypt(data, key):
    encrypted = []

    for i, char in enumerate(data):
        encrypted.append(
            chr(ord(char) ^ ord(key[i % len(key)]))
        )

    return ''.join(encrypted)


# Decryption function
def decrypt(data, key):
    return encrypt(data, key)


# Create an IP packet
source_ip = "192.168.1.10"
destination_ip = "10.0.0.5"
data = "Confidential Network Data"

ip_packet = (
    "Source IP: " + source_ip +
    "\nDestination IP: " + destination_ip +
    "\nData: " + data
)

print("Original IP Packet:")
print(ip_packet)


# IPSec Tunnel Mode
# Entire IP packet is encrypted
encrypted_packet = encrypt(ip_packet, key)

# Encode for safe display/transmission
encoded_packet = base64.b64encode(
    encrypted_packet.encode('latin1')
).decode()

print("\nIPSec Tunnel Mode Packet:")
print(encoded_packet)


# Receiver side
decoded_packet = base64.b64decode(
```

```
    encoded_packet
).decode('latin1')

decrypted_packet = decrypt(
    decoded_packet,
    key
)

print("\nDecrypted IP Packet:")
print(decrypted_packet)


# Verification
if decrypted_packet == ip_packet:
    print("\nIPSec Tunnel Mode:")
    print("Encryption and decryption successful.")
else:
    print("\nIPSec Tunnel Mode:")
    print("Decryption failed.")
```

Sample Output

```
Original IP Packet:
Source IP: 192.168.1.10
Destination IP: 10.0.0.5
Data: Confidential Network Data

IPSec Tunnel Mode Packet:
...

Decrypted IP Packet:
Source IP: 192.168.1.10
Destination IP: 10.0.0.5
Data: Confidential Network Data

IPSec Tunnel Mode:
Encryption and decryption successful.
```

Difference Between Tunnel Mode and Transport Mode

Feature	Tunnel Mode	Transport Mode
Encryption	Entire original IP packet	Only the IP payload
Original IP Header	Encrypted	Remains visible
New IP Header	Added	Not normally added
Main Usage	VPN and gateway-to-gateway communication	Host-to-host communication
Security	Provides greater protection for the original packet	Provides protection mainly for the data

Analysis

In IPSec Tunnel Mode, the **entire original IP packet**, including its original IP header and payload, is encrypted and encapsulated inside a new IP packet. This provides better protection for the original packet and is commonly used in VPN communication.
In Transport Mode, only the payload of the IP packet is protected while the original IP header remains visible. Therefore, Tunnel Mode provides greater privacy for the original IP packet.

Question 4: SSL/TLS Handshake Simulation

Aim

To write a Python program to simulate an SSL/TLS handshake, including certificate exchange, session key generation, and secure data transfer.

Algorithm

1. The client sends a connection request to the server.
2. The server responds with its digital certificate.
3. The client verifies the server certificate.
4. A session key is generated for secure communication.
5. The session key is used for symmetric encryption.
6. The client encrypts the data and sends it to the server.
7. The server decrypts the received data using the session key.
8. Display the secure communication between the client and server.

Python Program

```
import hashlib
import base64

# Simulated server certificate
server_certificate = {
    "server": "secure.example.com",
    "issuer": "Trusted Certificate Authority",
    "valid": True
}

# Session key
session_key = "TLS_SESSION_KEY"


# Encryption function
def encrypt(data, key):
    encrypted = []

    for i, char in enumerate(data):
        encrypted.append(
            chr(ord(char) ^ ord(key[i % len(key)]))
        )

    return ''.join(encrypted)


# Decryption function
```

```
def decrypt(data, key):
    return encrypt(data, key)


# Generate certificate fingerprint
def generate_fingerprint(certificate):
    certificate_data = (
        certificate["server"] +
        certificate["issuer"]
    )

    return hashlib.sha256(
        certificate_data.encode()
    ).hexdigest()


# ----------------------------
# SSL/TLS Handshake
# ----------------------------

print("===== SSL/TLS HANDSHAKE SIMULATION =====")

# Step 1: Client Hello
print("\n1. Client Hello")
print("Client requests a secure connection.")

# Step 2: Server Hello
print("\n2. Server Hello")
print("Server accepts the secure connection.")

# Step 3: Certificate Exchange
print("\n3. Certificate Exchange")
print("Server:", server_certificate["server"])
print("Issuer:", server_certificate["issuer"])

# Generate certificate fingerprint
fingerprint = generate_fingerprint(
    server_certificate
)

print("Certificate Fingerprint:")
print(fingerprint)

# Step 4: Certificate Verification
print("\n4. Certificate Verification")

if server_certificate["valid"]:
    print("Certificate verified successfully.")
```

```
else:
    print("Certificate verification failed.")
    exit()

# Step 5: Session Key Generation
print("\n5. Session Key Generation")
print("Session key generated successfully.")

# Step 6: Secure Data Transfer
message = input("\nEnter message to send securely: ")

encrypted_message = encrypt(
    message,
    session_key
)

encoded_message = base64.b64encode(
    encrypted_message.encode('latin1')
).decode()

print("\nEncrypted Data:")
print(encoded_message)

# Step 7: Server decrypts the data
decoded_message = base64.b64decode(
    encoded_message
).decode('latin1')

decrypted_message = decrypt(
    decoded_message,
    session_key
)

print("\nDecrypted Data at Server:")
print(decrypted_message)

# Final result
if decrypted_message == message:
    print("\nSecure Data Transfer Successful.")
else:
    print("\nSecure Data Transfer Failed.")
```

Sample Output

```
===== SSL/TLS HANDSHAKE SIMULATION =====

1. Client Hello
Client requests a secure connection.

2. Server Hello
```

Server accepts the secure connection.

3. Certificate Exchange
Server: secure.example.com
Issuer: Trusted Certificate Authority

Certificate Fingerprint:
7f4c...

4. Certificate Verification
Certificate verified successfully.

5. Session Key Generation
Session key generated successfully.

Enter message to send securely: Hello Secure Server

Encrypted Data:
...

Decrypted Data at Server:
Hello Secure Server

Secure Data Transfer Successful.

Analysis

The SSL/TLS handshake establishes a secure communication session between the client and server. The server provides a digital certificate that is verified by the client. After successful verification, a session key is generated and used for secure data transfer.

The handshake provides **authentication** through certificate verification and **confidentiality** through encryption of the transmitted data. Using a session key also allows efficient symmetric encryption during the communication session.

Question 5: Email Phishing Detection

Aim

To build a Python-based tool to detect phishing emails using feature extraction based on URL patterns, headers, and keywords, and to analyze the effectiveness of the detection method.

Algorithm

1. Read the email content.
2. Extract features such as suspicious URLs, phishing-related keywords, and header information.
3. Assign a score based on the number of suspicious features.
4. Compare the score with a predefined threshold.
5. Classify the email as phishing or legitimate.
6. Display the detected features and classification result.

Python Program

```
import re

# Phishing keywords
phishing_keywords = [
    "urgent",
    "verify your account",
    "click here",
    "password",
    "login",
    "confirm your account",
    "bank",
    "suspended",
    "winner",
    "free"
]


# Detect suspicious URLs
def detect_urls(email):
    urls = re.findall(
        r'https?://[^\s]+',
        email
    )

    suspicious_urls = []

    for url in urls:
        if (
            "@" in url or
            "bit.ly" in url or
            "tinyurl" in url or
            url.count("-") >= 2
        ):
            suspicious_urls.append(url)

    return suspicious_urls


# Detect phishing keywords
def detect_keywords(email):
    email_lower = email.lower()

    found_keywords = []

    for keyword in phishing_keywords:
        if keyword in email_lower:
            found_keywords.append(keyword)
```

```
    return found_keywords


# Phishing detection
def detect_phishing(email):
    score = 0

    suspicious_urls = detect_urls(email)
    found_keywords = detect_keywords(email)

    # URL score
    score += len(suspicious_urls) * 2

    # Keyword score
    score += len(found_keywords)

    # Header analysis
    if "from:" not in email.lower():
        score += 1

    if "reply-to:" not in email.lower():
        score += 1

    # Classification
    if score >= 3:
        result = "PHISHING EMAIL"
    else:
        result = "LEGITIMATE EMAIL"

    return result, score, suspicious_urls, found_keywords


# Input email
print("===== EMAIL PHISHING DETECTOR =====")

email = input("\nEnter the email content:\n")

result, score, urls, keywords = detect_phishing(email)


# Display results
print("\n===== ANALYSIS RESULT =====")

print("Phishing Score:", score)

print("\nSuspicious URLs:")

if urls:
```

```
    for url in urls:
        print("-", url)
else:
    print("None")

print("\nSuspicious Keywords:")

if keywords:
    for keyword in keywords:
        print("-", keyword)
else:
    print("None")

print("\nClassification:")
print(result)
```

Sample Input

From: unknown@example.com
Reply-To: attacker@example.com

URGENT! Verify your account immediately.
Click here to confirm your account:
http://bit.ly/login

Sample Output

```
===== EMAIL PHISHING DETECTOR =====

===== ANALYSIS RESULT =====

Phishing Score: 6

Suspicious URLs:
- http://bit.ly/login

Suspicious Keywords:
- urgent
- verify your account
- click here
- confirm your account
- login

Classification:
PHISHING EMAIL
```

Analysis

The program extracts important features from an email, including suspicious URLs, phishing-related keywords, and email header information. Each suspicious feature contributes to a phishing score.

If the score reaches the predefined threshold, the email is classified as a phishing email. Otherwise, it is classified as legitimate.

This approach can identify common phishing patterns, but its effectiveness depends on the selected features and threshold. More advanced systems can use machine-learning classification techniques trained on larger datasets to improve detection accuracy.